

ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 20/04

- Estructuras de control repetitivas.
- Estructura WHILE
- Estructura FOR.
- Estructura DO WHILE.

2

AEDD – Tareas semanales!!!!

Vincularse al curso en Omega Up: Comision F-2026-Anual.

La tarea de esta semana se publica este viernes en esta plataforma y la deben cumplir todos los alumnos a mas tardar cada domingo antes de las 23:59 hs.

Esto nos sirve para llevar la materia al día, y además para ir evaluando que contenidos son comprendidos y cuales deben ser revisados.

3

Metodología de Programación Estructurada – Recordemos!

Metodología que consiste en escribir programas con las siguientes reglas:

- El programa tiene un diseño modular
- Los módulos son diseñados de modo descendente
- Cada módulo se codifica utilizando las tres estructuras de control básicas:
 - Secuencia
 - Selección
 - Repetición

4

AEDD – Asistencia del día 20/04/2026

1larak

5

Estructuras de control: Repetición

- Un bucle es una construcción que repite una sentencia, o una secuencia de sentencias, un número de veces.
- El fin del bucle DEBE alcanzarse.
- Se definen las siguientes partes de un bucle:
 - Cuerpo del bucle: la sentencia o grupo de sentencias que se repiten.
 - Condición del bucle: expresión lógica que controla la secuencia de repetición.
 - Iteración: Cada repetición del cuerpo del bucle.

6

Ciclo WHILE

Bucle WHILE – Bucle pretest – Bucle 0 hasta N

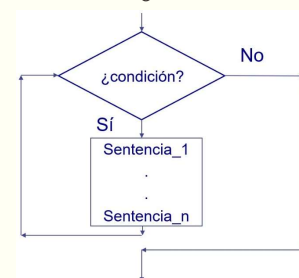
- La condición se evalúa antes de que se ejecute el cuerpo.
- La ejecución del cuerpo se repite mientras la condición del bucle sea verdadera y termina cuando se hace falsa
- Si la condición inicialmente es falsa el cuerpo no se ejecutará
- El bucle se ejecuta cero o más veces.

```
while (condición) {
    Sentencia 1;
    Sentencia 2;
```

```
....
}
```

Paréntesis ()
obligatorios

Las {} pueden
suprimirse si el
bloque contiene
1 sola sentencia



7

Ciclo WHILE

- La condición del bucle evalúa una variable, denominada variable de control del bucle.
- Los pasos son:
 1. Inicializar la variable de control antes de entrar al ciclo while
 2. Comprobar el valor de la variable de control antes de que comience la repetición del ciclo (denominada pasada o iteración)
 3. Actualizar la variable de control durante cada iteración.

8

Ciclo WHILE – Paso a paso su funcionamiento

1. Se evalúa la condición_bucle
2. Si la condición_bucle es verdadera (distinta de 0):
 - a) Se ejecuta la/s sentencia/s especificada/s en el cuerpo del bucle.
 - b) Se devuelve el control al paso 1.
3. Si la condición_bucle es falsa (igual a 0):
 - a) El control se transfiere a la sentencia siguiente al bucle while.

Las sentencias del cuerpo del bucle se repiten mientras que la condición_bucle sea verdadera.
 Cuando la condición_bucle es falsa se sale del bucle y se ejecuta la sentencia que viene después de la sentencia while (bloque while), en el programa

9

Ciclo WHILE – Ejemplo 1

1. Sumar los 10 primeros números naturales:

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int numero = 1, suma = 0;
6      while (numero <= 10) {
7          suma = suma + numero;
8          numero ++;
9      }
10     cout << "La suma de los 10 primeros numeros naturales es: " << suma;
11
12     return 0;
13 }
14

```

La suma de los 10 primeros numeros naturales es: 55

<< El programa ha finalizado: codigo de salida: 0 >>
 << Presione enter para cerrar esta ventana >>_

10

Ciclo WHILE – Ejemplo 2

Leer la cantidad de facturas emitidas en una empresa y a continuación los montos (valores decimales) de c/u e informar el total facturado y el mayor valor de una factura emitida.

¡¡A pensar!!

11

Ciclo WHILE – Ejemplo 2

Leer la cantidad de facturas emitidas en una empresa y a continuación los montos (valores decimales) de c/u e informar el total facturado y el mayor valor de una factura emitida.

```
#include <iostream>
using namespace std;

int main() {
    int cantFacturas, contador=1;
    float suma = 0, totalFactura, montoMayor;

    cout << "Ingrese la cantidad de facturas a procesar: ";
    cin >> cantFacturas;
    while (contador <= cantFacturas) {
        cout << "Ingrese el monto total de la factura numero " << contador << " : ";
        cin >> totalFactura;
        suma += totalFactura;

        if (totalFactura > montoMayor) montoMayor = totalFactura;
        contador ++;
    }
    cout << "El total facturado es: " << suma << endl;
    cout << "El monto mayor facturado es : " << montoMayor;
    return 0;
}
```

Ingrese la cantidad de facturas a procesar: 4
 Ingrese el monto total de la factura numero 1 : 124.55
 Ingrese el monto total de la factura numero 2 : 324.90
 Ingrese el monto total de la factura numero 3 : 1200.80
 Ingrese el monto total de la factura numero 4 : 3005.89
 El total facturado es: 4656.14
 El monto mayor facturado es : 3005.89

<< El programa ha finalizado: codigo de salida: 0 >>
 << Presione enter para cerrar esta ventana >>

12

AEDD – Asistencia del día 20/04/2026

llarak

13

Ciclo WHILE – CUIDADOS!

- Quedar en bucles infinitos, donde nunca la condición es falsa.
- O el caso contrario que nunca sea verdadera la condición, por lo que nunca se ejecuta el bucle.

Pensemos claramente la condición de ciclo while de forma de no caer en este tipo de problemas.

14

Ciclo WHILE con centinela

A veces no se conoce con exactitud cuantos datos se procesarán en el programa. Una forma de manejarlo es introducir como último dato un valor denominado valor centinela que indica que se ha finalizado la secuencia de números.

La condición del bucle comprueba cada dato y termina cuando se lee el valor centinela.

Modifiquemos el ejemplo 2, donde la carga de montos totales de facturas finaliza cuando ingresa como monto total = -1

¡¡A modificar el ejemplo 2, generando el ejemplo 3!
Manos a la obra

15

Ciclo WHILE con centinela

Modifiquemos el ejemplo 2, donde la carga de montos totales de facturas finaliza cuando ingresa como monto total = -1

The screenshot shows a C++ program in a code editor and its execution output in a console window. The program uses a while loop to process invoice amounts until a sentinel value of -1 is entered. It calculates the sum of all amounts and identifies the highest amount.

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int contador = 1;
6      float suma = 0, totalFactura = 0, montoMayor;
7      cout << "Ingrese el monto total de la factura numero " << contador << " : ";
8      cin >> totalFactura;
9
10
11     while (totalFactura != -1) {
12         suma += totalFactura;
13         contador++;
14         if (totalFactura > montoMayor) montoMayor = totalFactura;
15         cout << "Ingrese el monto total de la factura numero " << contador << " : ";
16         cin >> totalFactura;
17
18     }
19     if (contador > 1) {
20         cout << "El total facturado es: " << suma << endl;
21         cout << "El monto mayor facturado es: " << montoMayor;
22     }
23     return 0;
24 }

```

The console output shows the program running and the user entering values for four invoices. The final output displays the total amount factured (8443.31) and the highest amount factured (4000.66).

```

C:\Program Files (x86)\Zinjal\bin\runner.exe
Ingrese el monto total de la factura numero 1 : 1230.77
Ingrese el monto total de la factura numero 2 : 3211.88
Ingrese el monto total de la factura numero 3 : 4000.66
Ingrese el monto total de la factura numero 4 : -1
El total facturado es: 8443.31
El monto mayor facturado es: 4000.66
El programa ha finalizado: codigo de salida: 0 >>
Presione enter para cerrar esta ventana >>

```

16

Ciclo WHILE con bandera

- Se establece la variable de control (bandera) a verdadero o falso, según corresponda, para que el ciclo se ejecute al menos una vez.
- Mientras la condición de control sea verdadera se deben realizar las sentencias del ciclo y cuando se produzca la condición de salida, se cambia el valor de la bandera para que la próxima evaluación de la condición de control del ciclo sea falsa y se finalice el ciclo.
- Ejemplo: Solicitar al usuario que ingrese una letra minúscula.

```
int main(){
    int es_minus;
    char letra;
    es_minus = 0;
    while (!es_minus) {
        cout << endl << "Ingrese una letra minuscúla: ";
        cin >> letra;
        es_minus = (('a' <= letra) && (letra <= 'z'));
    }
    return 0;
}
```

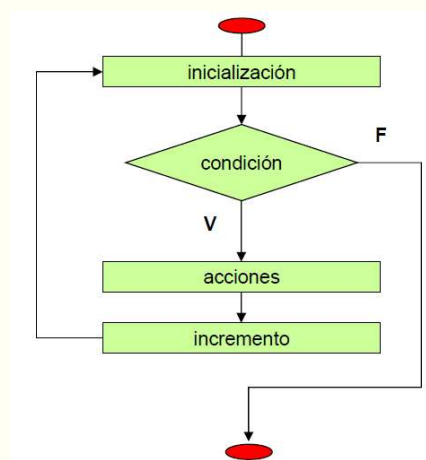
Valor bandera

Ingrese una letra minuscúla: A
 Ingrese una letra minuscúla: Z
 Ingrese una letra minuscúla: b

17

Ciclo FOR

- El bucle for: es un método para ejecutar un bloque de sentencias un número fijo de veces



18

Ciclo FOR

- Las operaciones de control del bucle se sitúan en un solo sitio: la cabecera de la sentencia for.
- En C++ su formato es:
 - inicialización: se inicializan las variables de control (simples o múltiples).
 - condición: expresión lógica que hace que el bucle realice las iteraciones mientras que la expresión sea verdadera.
 - incremento: incremento o decremento de las variables de control.

```
for (inicialización; condición; incremento) {  
  
    sentencias....  
  
}
```

Las { } pueden suprimirse si el bloque contiene 1 sola sentencia.

19

Ciclo FOR

- Ejemplo 4: Modificar el ejemplo 1, para trabajarlo con For. Sumar los 10 primeros números naturales

¡¡Manos a la obra!!

20

Ciclo FOR – Ejemplo 4

- Ejemplo 4: Modificar el ejemplo 1, para trabajarlo con For. Sumar los 10 primeros números naturales

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int suma = 0;
6      for (int numero = 1; numero <= 10; numero++) {
7          suma = suma + numero;
8      }
9      cout << "La suma de los 10 primeros numeros naturales es: " << suma;
10
11     return 0;
12 }
```

21

Ciclo FOR – Ejemplos para hacer en casa

- Ejemplo 5: Obtener el factorial de n.

- Ejemplo 6: Leer y sumar 10 notas.

22

Ciclo FOR – Ejemplos para hacer en casa

- Ejemplo 5: Obtener el factorial de n.

```
int main(){
    int n, factorial, x;
    cout << "Ingrese n: ";
    cin >> n;
    factorial = 1;
    for (x=2; x<=n; x++)
    {
        factorial = factorial * x;
    }
    cout << n << "!" = " << factorial;

    return 0;
}
```

```
Ingrese n: 3
3! = 6
```

- Ejemplo 6: Leer y sumar 10 notas.

```
int main(){
    int nota, sum, x;
    sum = 0;
    cout << "Ingrese 10 notas: " << endl;
    for (x=1; x<=10; x++){
        cin >> nota;
        sum +=nota;
    }
    cout << "La suma de las notas es: " << sum;
    return 0;
}
```

```
Ingrese 10 notas:
9
7
6
8
10
5
9
7
10
10
La suma de las notas es: 81
```

23

Ciclo FOR – Ejemplos

- Los incrementos / decrementos pueden ser enteros o reales, positivos o negativos.
- Mostrar los números múltiplos de 10, menores que 100, ordenados de mayor a menor:

```
int main(){
    int n;
    for (n=90; n>0; n-=10)
        cout << n << endl;
    return 0;
}
```

```
90
80
70
60
50
40
30
20
10
```

24

Ciclo FOR

- Es posible y adecuado declarar e inicializar la variable de control del bucle en el lugar de la inicialización.
- En este caso especial, el ámbito de visibilidad de la variable de control del bucle es solamente hasta el final del bloque de la estructura for.
- Ejemplo: Ingresar un número natural n y mostrar todos los naturales comprendidos entre 0 y el número ingresado:

```
#include <iostream>
using namespace std;

int main(){
    int n;
    cin >> n;
    for (int i=0 ; i < n ; ++i){
        cout << i << " ";
    }
    // i NO es visible aqui
    cout << endl;
    return 0;
}
```

25

Ciclo FOR: Ejemplo con múltiples variables de control

- Mostrar todos los pares de números naturales que suman un valor MAX

```
#define MAX 15

int main(){
    int i,j;
    for (i=0, j=MAX; i < j ; i++, j--)
        cout << "(" << i << ", " << j << ")" << endl;
    return 0;
}
```

```
(0,15)
(1,14)
(2,13)
(3,12)
(4,11)
(5,10)
(6,9)
(7,8)
```

26

Ciclo FOR - PRECAUCIONES

- **QUE TERMINE:** La inicialización, comprobación e incremento deben asegurar que la condición en algún momento sea falsa.
- Si la variable de control se modifica dentro del cuerpo del for, entonces esto no podrá asegurarse, por lo que se considera una mala práctica de programación.
- **CUIDADO CON LOS BUCLES VACÍOS.**

27

Ciclo DO-WHILE

El bucle do-while: se utiliza para especificar un bucle condicional que se ejecuta al menos una vez (ciclo 1-n veces).

Primero se ejecutan las sentencias, luego se evalúa la expresión, si es verdadera se repite la ejecución del ciclo y el proceso continúa hasta que sea falsa.

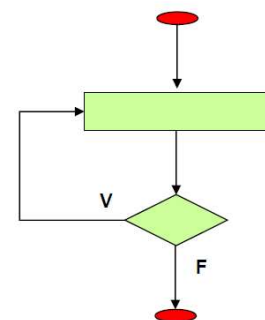
■ **En C++:**

```
do {
    sentencias;
} while (condición);
```

Las { } pueden suprimirse si el bloque contiene una sola sentencia.

El ; es obligatorio

Condición que debe cumplirse para iterar (verdadera)



28

Ciclo DO-WHILE

- Ejemplo 7: Imprimir todas las letras mayúsculas.

```
int main() {
    char car = 'A';
    do {
        cout << car << endl;
        car++;
    } while (car <= 'Z') ;

    return 0;
}
```



29

Comparación de ciclos: WHILE , FOR, DO-WHILE

- C++ brinda 3 sentencias para el control de bucles: while, for y do-while.

While	For	Do-while
Comienza su ejecución y se repite <i>mientras</i> su condición de repetición del bucle es verdadera.	Se utiliza cuando el número de repeticiones se conoce en forma exacta por anticipado.	Se ejecuta una vez y luego se repite <i>mientras</i> la condición de repetición del bucle sea verdadera.
El número de repeticiones depende de una condición (verdadera o falsa) → se realiza de 0 a n veces.	El número de repeticiones está establecido de antemano → se realiza N veces	El número de repeticiones depende de una condición (verdadera o falsa) → se realiza de 1 a n veces.
La condición está en la cabecera del bucle.	La condición está en la cabecera del bucle.	La condición está al final del bucle.
Puede ejecutarse 0 veces.	Puede ejecutarse 0 veces	Se ejecuta 1 vez como mínimo.
Es la más general.	Es una estructura muy flexible en C (no así en otros lenguajes).	Se puede simular con un while o un for.

30

Ciclos anidados

- En C++ es posible anidar bucles.
- Los bucles anidados constan de un bucle externo, con uno o más bucles internos.
- Cada vez que se repite el bucle externo, los bucles internos se repiten, se vuelven a evaluar los componentes de control y se ejecutan todas las iteraciones requeridas.

```
int main(){
    for (int x=1; x<=10; x++)
    {
        for (int y=1; y<=10; y++)
        {
            int producto;
            producto = x * y;
            cout << x << " * " << y << " = " << producto;
            cout << endl;
        }
    }
    return 0;
}
```

```
1*1=1
1*2=2
1*3=3
1*4=4
1*5=5
1*6=6
1*7=7
1*8=8
1*9=9
1*10=10
2*1=2
2*2=4
2*3=6
2*4=8
2*5=10
2*6=12
2*7=14
2*8=16
2*9=18
2*10=20
3*1=3
3*2=6
3*3=9
3*4=12
3*5=15
```

31

Ejercicio 1

Dado un email: usuario@dominio.com

- Realizar:
- Separar:
 - usuario
 - dominio
- Validar:
 - Que exista exactamente un @
 - Que haya al menos un . después del @
- Mostrar:
 - Usuario
 - Dominio
 - Extensión (lo que está después del último .

32

Ejercicio 2

Dado un string ingresado por el usuario:

1. Eliminar todos los signos de puntuación (.,:;!?)
2. Convertir la primera letra de cada palabra a mayúscula y el resto a minúscula
3. Mostrar:
 1. La frase normalizada
 2. La cantidad de palabras

33

Ejercicio 3

Dado un texto con múltiples espacios:

- "Hola mundo C++"
- Convertirlo en:
- "Hola mundo C++"

34

AEDD – Asistencia del día 20/04/2026

llarak

35

AEDD – Tareas semanales!!!!

Vincularse al curso en Omega Up: Comision F-2026-Anual.

La tarea de esta semana se publica este viernes en esta plataforma y la deben cumplir todos los alumnos a mas tardar cada domingo antes de las 23:59 hs.

Esto nos sirve para llevar la materia al día, y además para ir evaluando que contenidos son comprendidos y cuales deben ser revisados.

36

AEDD – Otras tareas !!!!!

Leer Capítulo 4
Libro:
Benjumea-Roldan
Univ-de-Málaga

37

Algoritmos y Estructuras de Datos – AEDD – Agenda 20/04

- ✓ Estructuras de control repetitivas.
- ✓ Estructura WHILE
- ✓ Estructura FOR.
- ✓ Estructura DO WHILE.

¡¡OBJETIVOS DEL DIA ALCANZADOS!!

38

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



39

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Buena semana de cursado!



40